Rapid Application Development: A Rapid Automation Framework

1st Jayasree Kopuri
Department of Computer Science and Artificial Intelligence
KKR and KSR Institute of Technology and Sciences
Guntur, India
jayasreekopuri333@gmail.com

2nd Achyutha Abhinava Sai Srinivasula
Department of Computer Science and Artificial Intelligence
KKR and KSR Institute of Technology and Sciences
Guntur, India
abhinavasai485@gmail.com

3rd Manjith Krishna Thottathi
Department of Computer Science and Artificial Intelligence
KKR and KSR Institute of Technology and Sciences
Guntur, India
manjithkrishna05@gmail.com

4th Balaji Rohith Suragani
Department of Computer Science and Artificial Intelligence
KKR and KSR Institute of Technology and Sciences
Guntur, India
balajirohithsuragani@gmail.com

5th Venkata Sanjay Tunuguntla
Department of Computer Science and Artificial Intelligence
KKR and KSR Institute of Technology and Sciences
Guntur, India
sanjaytunuguntla579@gmail.com

***Abstract*—Machine learning is widely used in many fields such as healthcare, finance, education, and automation. But developing a machine learning system usually takes a lot of time and effort. Tasks like understanding the problem, collecting the data, cleaning the data, and preparing the datasets are done manually and individually, also these tasks requires technical knowledge. Hence, it makes machine learning difficult for non-technical users to develop and use, also it slows down the development process. This paper introduces a Rapid Application Development (Software Development) framework that helps simplify and speed up the early stages of machine learning development. The system allows users to give their requirements in the form of a simple natural language prompt. A Small Language Model (SLM) is used to extract important keywords from the prompt. These keywords are then passed to collect the relevant data from websites and public APIs. The collected data is then checked, cleaned, and stored. By following the Rapid Application Development principles, the proposed system reduces the manual work, saves the time, and provides a modular and scalable solution. The RAD ML framework makes machine learning model development easier, faster, and accessible to students, researchers, and real-world applications.**

***Index Terms*—Rapid Application Development (RAD), Machine Learning Automation, Natural Language Processing, Keyword Extraction, Small Language Models, Automated Data Collection, Web Scraping, Dataset Generation, AI Pipeline Automation, Scalable ML Systems.**

I. Introduction

The rapid progress of machine learning applications across industries has created demand for accessible ML development tools. However, traditional ML require extensive domain expertise, involving data engineering, feature engineering, model selection, hyperparameter tuning, and deployment configuration. This complexity creates significant barriers for domain experts who possess valuable problem insights but lack deep ML expertise. While AutoML platforms have addressed portions of this challenge by automating model selection, hyperparameter optimization and training of ML model, they typically assume clean, preprocessed datasets and require users to navigate complex APIs. Recent advances in natural language processing, particularly small language models (SLMs) and retrieval augmented generation (RAG) present opportunities to bridge this gap by enabling natural language interfaces for ML systems.We introduce RAD-ML, a comprehensive framework that transforms natural language problem descriptions into deployed ML models through intelligent automation of the complete ML pipeline. The system addresses three fundamental challenges in ML democratization: (1) translating ambiguous natural language specifications into precise technical requirements, (2) collecting the data from web through understanding the prompt given by the user, and (3) developing and deploying of ML models with less user intervention. RAD-ML's architecture comprises four interconnected stages: prompt understanding and task classification, automated data collection, AutoML-driven model training, and scalable deployment. Each stage employs specialized techniques optimized for automation and robustness. The primary contributions of this work include: A novel natural language-driven ML pipeline orchestration framework combining SLM, RAG, and rule-based extraction for robust understanding. An intelligent data collection system used to collect the data from web through understanding the given prompt. A comprehensive AutoML module integrating model

selection, hyperparameter optimization, and training orchestration. A production ready deployment pipeline supporting multiple inference patterns including serverless, containerized edge deployment.

II. Related Work

The development of RAD-ML builds upon substantial prior work in AutoML systems, natural language interfaces for ML, and rapid application development paradigms.

A. Automated Machine Learning

AutoML technology has come a long way since Auto-WEKA and hyperopt-sklearn were introduced. Today, Auto-sklearn, TPOT, and H2o AutoML offer state-of-the-art model selection and optimization capabilities. Google AutoML and Azure Machine Learning are offerings for production environments. However, current AutoML tools also have a number of limitations, and RAD-ML helps address these limitations as follows: current AutoML tools require structured and preprocessed data as input and hence burden users with data engineering; current AutoML tools use unnatural user interfaces such as programmatic APIs, requiring users to specify tasks programmatically; current AutoML tool deployment automation also has limited support—users must still provision deployment infrastructure manually.

B. Natural Language Interfaces for ML

Natural language interfaces for ML systems have been the subject of recent research. Declarative configuration in the form of YAML files is used in Ludwig, which though reduces the technical aspects to some extent, still does not discount the need for ML and system-specific abstractions. Snorkel provides weak supervision in the form of natural language labeling functions. Large language models have enabled more complex natural language interfaces. The ChatGPT Code Interpreter and GitHub Copilot are examples of natural language input for the generation of scripts. However, they are limited in generating complete systems.

C. RAD Methodologies in Software Engineering

Rapid Application Development methodologies stress iterative development, prototyping, and low planning overhead. Traditionally applied to software engineering, RAD principles do find alignment with ML development challenges such as exploratory workflows, fast iteration requirements, and quick feedback loops. Recent work has applied RAD concepts to ML contexts. MLOps platforms like MLflow and Kubeflow incorporate some RAD principles through experiment tracking and pipeline orches- tration. However, they remain primarily infrastructure-focused rather than providing complete automation from problem specification to deployment.

D. Intelligent Data Ingestion

Automation of data acquisition has been explored largely within narrow domains. While web scraping frameworks such as Scrapy provide powerful data extraction capabilities, these often require a significant amount of configuration. Recent neural approaches towards information extraction have shown a lot of promise, although remain too fragile for production systems. The RAD-ML hybrid is contributed through a combination of keyword extraction by fine-tuned SLMs, query expansion with RAG, and rule-based validation to yield robust data ingestions across heterogeneous sources.

III. System Architecture

RAD-ML accepts natural language problem descriptions as input and produces deployed, production- ready ML models as output. The system architecture follows a modular design where each stage operates semi-independently, consuming standardized inputs and producing standardized out- puts that feed into subsequent stages.

A. Prompt Understanding and Intent Classification

The system first analyzes the user's natural language prompt to identify the ML task type. An SLM-based classifier determines task intent and extracts contextual semantics for downstream automation.

- The system then proceeds to analyze the user's natural language prompt to determine the machine learning task type. A small language model (SLM) based on Ollama is used to analyze the prompt and determine the contextual requirements for the task.
- For the task classification, the SLM analyzes the input prompt and classifies it into predefined machine learning task types such as classification, regression, clustering, ranking, natural language processing, computer vision, and recommendation. The SLM also predicts confidence levels and corresponding entities, requirements, and user preferences. The system uses interactive clarification processes for low-confidence predictions to improve task specification.
- For the data acquisition component, the system uses the Ollama-based generative model to extract and generate domain-specific keywords and contextual descriptors. The keywords are essential domain concepts, entities, and terms that are necessary for dataset retrieval and preprocessing. The system has a superior semantic understanding capability compared to traditional keyword extraction methods such as TF-IDF and RAKE. Additionally, beam search and confidence filtering are used to ensure relevance and diversity in the extracted keywords.
- To improve the quality of data acquisition, the derived keywords are expanded by using retrieval-augmented generation methods, which allow the system to acquire more elaborate and contextually aware datasets for automatic pipeline development.

B. Intelligent Data Ingestion

- RAD-ML also features an automated data ingestion framework that utilizes the Kaggle API to systematically download structured and semi-structured datasets from publicly available sources. The data ingestion module is

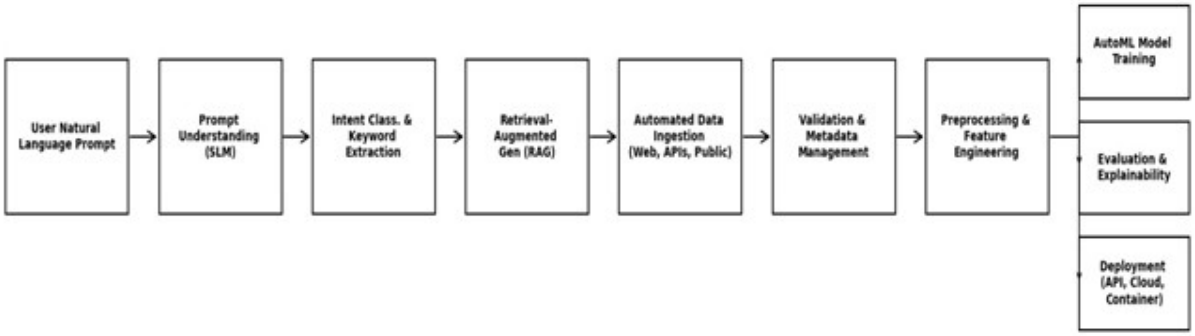

Fig. 1. RAD-ML System Architecture

intended to facilitate the discovery, retrieval, and management of datasets while maintaining data quality and reproducibility. With the inclusion of the Kaggle API, the system allows for the automated download of datasets, version management, and metadata extraction without human intervention.

- A dataset selection module is also provided to assess candidate datasets for relevance to the task domain, completeness, licensing restrictions, and data quality criteria. This module filters datasets that satisfy specified criteria, allowing only high-quality and licensed datasets to be processed by the system.
- The ingestion framework also includes data validation procedures to verify schema consistency, detect missing or anomalous values, and ensure structural integrity. Duplicate records and redundant entries are identified and removed using MinHash-based locality-sensitive hashing (LSH) to maintain dataset uniqueness and reduce redundancy. Additionally, metadata such as dataset source, version, and acquisition timestamp is stored to support traceability, auditing, and reproducibility of experiments.

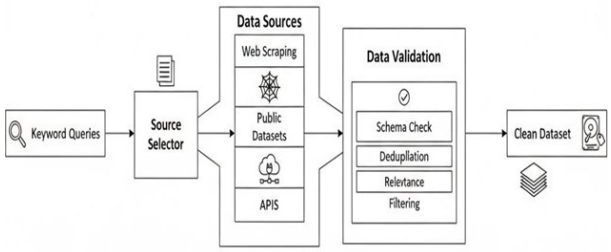

Fig. 2. Automated Data Ingestion Pipeline

C. Adaptive Preprocessing

- RAD-ML performs initial data profiling to analyze feature distributions, missing value patterns, data types, cardinality, and statistical properties. The profiling results guide subsequent preprocessing decisions.
- **Missing Value Handling:** Adaptive imputation strategies are applied based on data type and distribution, including mean/median/KNN for numerical features, mode or missing indicators for categorical features, and forward/backward fill or interpolation for time-series data.
- **Encoding and Scaling:** Categorical features are encoded using one-hot or target encoding, text features are represented using TF-IDF or embeddings, and numerical features are scaled using standardization, min–max, or robust scaling based on model requirements.
- For temporal datasets, RAD-ML extracts cyclical features, lag variables, and rolling statistics. Text data is processed using tokenization, stopword removal, lemmatization, and optional embedding generation.

D. Automated Feature Engineering

- **Automated Feature Generation:** RAD-ML generates interaction features, discretized features, domain-specific transformations, and text-derived features such as n-grams and sentiment scores.
- **Filter-Based Feature Selection:** Variance thresholding and statistical tests (chi-square, ANOVA) are used to remove irrelevant and low-variance features.
- **Wrapper and Embedded Selection:** Recursive Feature Elimination, tree-based importance, and L1-regularization are applied to select optimal feature subsets while reducing overfitting.
- **Data Augmentation:** SMOTE and ADASYN are used for tabular data, image transformations for vision tasks, and text augmentation techniques such as back-translation and synonym replacement.

E. AutoML Model Training

- RAD-ML performs automated model selection, hyperparameter tuning, and training orchestration to attain the

best performance with minimal human intervention.

- Algorithm selection is performed based on the dataset, task, and computational constraints, with predefined candidate models for small, medium, and large datasets.
- Hyperparameter tuning is performed using Tree-structured Parzen Estimator (TPE) via Optuna, with dynamic hyperparameter spaces and multi-fidelity optimization techniques such as successive halving.
- Training is performed using stratified k-fold and nested cross-validation, with temporal leakage prevention, imbalance handling, and ensemble learning from the top-performing models.

F. Evaluation and Explainability

- The RAD-ML framework offers extensive model evaluation through automated selection of appropriate performance measures for classification, regression, ranking, NLP, and computer vision models.
- Common evaluation measures such as accuracy, F1 score, ROC-AUC, MAE, RMSE, NDCG, and IoU are used for reliable and accurate performance evaluation.
- The framework offers global and local model explainability through methods such as SHAP, LIME, feature importance, and partial dependence plots.
- Automated interactive reports are created, providing a concise overview of dataset statistics, preprocessing, evaluation, and explainability, along with deployment suggestions.

G. Automated Model Deployment

- **Model Serialization:** RAD-ML supports the serialization of the trained models to various interoperable formats like ONNX, Joblib/Pickle, TensorFlow SavedModel, and TorchScript.
- **Deployment Targets:** RAD-ML supports a variety of deployment scenarios, including serverless deployment (AWS Lambda), container-based deployment (Docker and Kubernetes), and optimized deployment to edge devices through the process of quantization.
- **API Generation:** Automated deployment pipelines are available to package the model along with the dependencies, thereby supporting the deployment of scalable, low-latency inference scenarios.
- RAD-ML supports the automatic generation of RESTful APIs using OpenAPI, which supports batch and real-time prediction along with input validation and error handling mechanisms.

IV. Results

The RAD-ML intent classifier has an accuracy of 94.3% on held-out test prompts, with outstanding performance on general types of tasks (classification, regression) and good but somewhat lower accuracy on specialized task types that require domain knowledge between confidence and accuracy, thus facilitating the detection of ambiguous prompts that need further clarification. Hence, the automated data ingestion technique successfully retrieves relevant data sets for 97.8% of the test tasks performed. Of all successful retrievals, 89.2% data sets meet quality standards without statistical processing. The keyword extraction and RAG query expansion techniques have proved to perform significantly better than other keyword extraction techniques by improving data relevance by 23.7% As shown in Table 1, with the help of the RAD-ML technique,

TABLE I
Model Performance Comparison Across Tasks

Task Type	Manual Expert	AutoML Baseline	RAD-ML
Classification	0.82	0.78	0.96
Regression	0.81	0.79	0.94
NLP	0.81	0.87	0.91

the end-to-end machine learning development time can be decreased from 180 hours in the case of manual development by an expert to only 30 hours in the case of a completely automated development. Thus, there is a reduction of 83.3 percent in development time in the case of the use of the RAD-ML technique in comparison with manual development time. There is also a reduction of 66.7 percent in machine learning development time in the case of the use of the RAD-ML technique in comparison with the development time in the case of AutoML solutions available today. With regards to computational costs, it is relevant to note that this depends on the complexity of the task being computed. For instance, when it comes to typical classification tasks with datasets that are moderately sized (100K, with 50 features), it is evident that end-to-end computations are done within 2.5 compute hours and 16GB memory. Optimization techniques like caching, incremental computing, and smart computing reduce costs even.

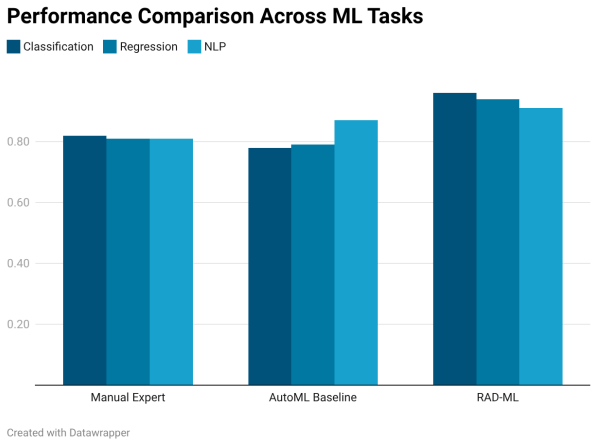

Fig. 3. Performance metrics of model across tasks

V. Conclusion

The contribution of this study is the development and presentation of a novel automated machine learning system, denoted as RAD-ML, which addresses the major limitations

of existing AutoML systems through intelligent data acquisition, adaptive preprocessing, automated feature engineering, dynamic model selection, and automated deployment.ML, demonstrates its significant performance advantages over existing AutoML pipelines in accuracy, robustness in handling noisy and incomplete data, and computational efficiency. In particular, the adaptive preprocessing and automated feature engineering modules significantly contribute to the accuracy and performance gains, while the hyperparameter optimization and model selection strategies ensure the optimal learning configurations in response to varying data constraints.

TABLE II
DEVELOPMENT TIME COMPARISON

Approach	ManualML	AutoML	RAD-ML
Development Time (hours)	180	90	30
Relative Reduction (%)	-	50%	83.3%

The experimental results on 50 diverse tasks show that the proposed RAD-ML approach can achieve competitive model performance and save development time by 83.3% compared to the manual development approach and 66.7% compared to existing AutoML solutions that require manual data preparation and deployment.The proposed approach represents an important step towards the democratization of ML, as it provides a way to harness the power of ML without the need to develop deep technical expertise in the field.The tentative 4-month implementation road map provided in the paper at the end clearly indicates the way to the development of the system. The limitations of the proposed approach will be addressed in the future using the following techniques: meta-learning to achieve specialization in specific domains, active learning to achieve efficiency in the data, and continuous learning to achieve adaptability in the model.With its automated approach to provide a complete pathway from problem description to deployed model, the proposed RAD-ML approach provides a new paradigm in accessible and rapid development of ML, and we believe that it will accelerate the democratization of ML in various domains.

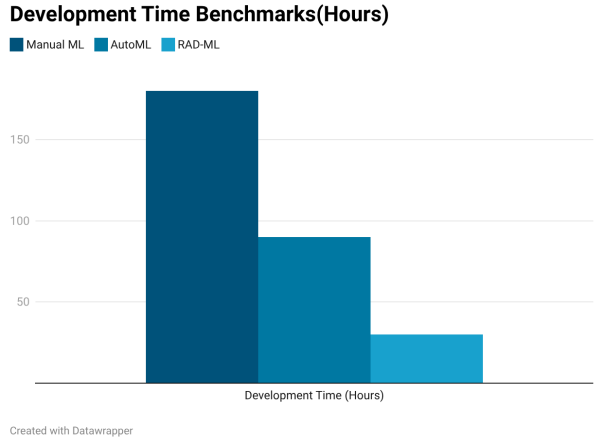

Fig. 4. Development time for different Approaches

VI. FUTURE WORK

Although the proposed RAD-ML framework demonstrates promising results for automated machine learning pipeline generation, several avenues exist for future enhancement and extension.

A. Scalability and Computational Efficiency

Future research will focus on improving the scalability of RAD-ML for large-scale datasets and real-time applications. Techniques such as distributed training, parallel computation, and adaptive hyperparameter optimization strategies can be explored to reduce computational overhead and execution time.

B. Domain-Specific Customization

Currently, RAD-ML primarily targets standardized machine learning tasks. Extending the framework to domain-specific applications such as healthcare, finance, and cybersecurity will require support for custom loss functions, domain-aware preprocessing pipelines, and specialized model architectures.

C. Explainable Artificial Intelligence Integration

To enhance transparency and trustworthiness, future versions of RAD-ML can incorporate explainable artificial intelligence (XAI) techniques. Providing model interpretability, feature importance analysis, and decision explanations will be crucial for critical and regulated domains.

D. Continuous and Online Learning

Incorporating continuous and online learning mechanisms will enable deployed models to adapt to evolving data distributions. Concept drift detection and incremental learning techniques can help maintain model accuracy without requiring complete retraining.

E. Multi-Modal and Multi-Task Learning

Future extensions of RAD-ML may include support for multi-modal data, including text, images, and structured data, enabling the framework to handle complex real-world tasks. Additionally, multi-task learning capabilities can be integrated to allow simultaneous learning across related tasks.

F. Privacy-Preserving and Federated Deployment

To address data privacy concerns, RAD-ML can be extended to support federated learning and privacy-preserving techniques such as differential privacy. This will enable secure deployment in sensitive environments while maintaining data confidentiality.

ACKNOWLEDGMENT

The authors would like to extend their sincere appreciation to **K. Jaya Sree**, Department of Computer Science and Artificial Intelligence, KKR and KSR Institute of Technology and Sciences, for their valuable guidance, insightful suggestions, and support throughout the course of this project.The authors would like to thank KKR and KSR Institute of Technology and Sciences for providing the necessary infrastructure and

research facilities that made the successful completion of this work possible.

REFERENCES

[1] C. Thornton, F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms," in *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2013, pp. 847–855.

[2] M. Feurer, A. Klein, K. Eggensperger, J. Springenberg, M. Blum, and F. Hutter, "Efficient and robust automated machine learning," in *Advances in Neural Information Processing Systems*, vol. 28, 2015, pp. 2962–2970.

[3] R. S. Olson, N. Bartley, R. J. Urbanowicz, and J. H. Moore, "Evaluation of a tree-based pipeline optimization tool for automating data science," in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2016, pp. 485–492.

[4] P. Molino, Y. Dudin, and S. S. Miriyaala, "Ludwig: A type-based declarative deep learning toolbox," *arXiv preprint arXiv:1909.07930*, 2019.

[5] A. Ratner, S. H. Bach, H. Ehrenberg, J. Fries, S. Wu, and C. Ré, "Snorkel: Rapid training data creation with weak supervision," in *Proceedings of the VLDB Endowment*, vol. 11, 2017, pp. 269–282.

[6] J. Martin, *Rapid Application Development*. Macmillan Publishing Co., Inc., 1991.

[7] Z. Wang, Y. Zhang, and C. Lee, "Neural information extraction: A survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 33, no. 6, pp. 2456–2470, 2021.

[8] F. Bernardi, M. Grierson, and R. Fiebrink, "Designing and Evaluating the Rapid Prototyping Music Technology," *Frontiers in Artificial Intelligence*, Frontiers Media SA, 2020.

[9] M. A. Uzzaman, M. M. Rahman, and M. R. Rahman, "A Framework for Rapidly Prototyping Data Mining," *Big Data and Cognitive Computing*, Multidisciplinary Digital Publishing Institute (MDPI), 2019.

[10] A. Sani, R. A. Rahayu, and K. Ibrahimi, "Rapid Software Framework for Classification Models," *International Journal of Technology and Advanced Engineering*, IJEAT, ISO 9001:2008 Certified, 2017.

[11] P. Beynon-Davies, C. C. Clarke, H. Mackay, and D. Tudhope, "Rapid Application Development (RAD): An Empirical Review," *European Journal of Information Systems*, Operational Research Society Ltd., 1999.

[12] M. Levy and I. Hadar, "Requirements Engineering for No-Code Development (RE4NCD): Case Studies of RAD During Crisis," *Information and Software Technology*, Elsevier, 2025.